

SENAI

Relatório Técnico

Vue.js

Um Framework Front-end para Construção de Interfaces Web

Atividade 02 — Desenvolvimento Web

Disciplina: Desenvolvimento de Sistemas Web

Integrantes: Rafaela Silva da Paz, João Alexandre da Silva Pereira, Marco Antônio da Silva Costa, Luís Felipe Santos Correia e Tuanny Cristina Thomazelli

2026

Sumário

1. Introdução
2. O que é o Vue.js
3. Principais Características
 - 3.1 Componentização
 - 3.2 Sistema de Reatividade
 - 3.3 Templates e Single-File Components
 - 3.4 Diretivas e Vinculação de Dados
 - 3.5 Composition API
 - 3.6 Ecossistema e Ferramentas
4. Vantagens do Vue.js
5. Desvantagens e Pontos de Atenção
6. Aplicações no Mercado

Tecnologia	Categoria	Principal característica	Perfil de uso
Vue.js	Framework Front-end	Reatividade e componentização	Interfaces e aplicações Web
React	Biblioteca UI	Componentização e ecossistema amplo	Interfaces e aplicações Web
Angular	Framework Front-end	Solução mais completa e estruturada	Aplicações corporativas de grande porte
Svelte	Framework/compiler	Compilação e abordagem reativa	Interfaces e aplicações Web

Tabela 5 — Comparação resumida entre tecnologias Front-end.

7. Exemplo Prático de Utilização em um Projeto Web
 - 7.1 Estrutura do Projeto
 - 7.2 Componente de Lista de Tarefas
 - 7.3 Explicação do Código
8. Conclusão
9. Referências

1. Introdução

O desenvolvimento de interfaces para a Web evoluiu significativamente nos últimos anos, passando de páginas estáticas construídas apenas com HTML e CSS para aplicações complexas, interativas e altamente dinâmicas. Esse cenário impulsionou o surgimento de bibliotecas e frameworks Front-end que facilitam a organização, a manutenção e a escalabilidade do código no lado do cliente.

Neste relatório técnico, o grupo apresenta o Vue.js, um framework JavaScript progressivo utilizado para a construção de interfaces de usuário. Serão abordadas suas principais características, vantagens, desvantagens, aplicações no mercado de trabalho e um exemplo prático de utilização em um projeto Web, com o objetivo de demonstrar, na teoria e na prática, como essa tecnologia pode ser empregada no desenvolvimento de aplicações modernas. O estudo também apresenta exemplos de organizações que utilizam Vue em produção e uma comparação de suas características com outras tecnologias Front-end, permitindo compreender melhor sua presença no mercado.

2. O que é o Vue.js

O Vue.js é um framework JavaScript de código aberto criado por Evan You e voltado para a construção de interfaces Web. O projeto é progressivo e pode ser adotado de maneira incremental, desde uma pequena parte de uma página até aplicações Single-Page Application (SPA) mais completas. A documentação oficial destaca como características centrais a facilidade de adoção, a renderização reativa e a organização por componentes.

Diferentemente de uma biblioteca focada somente em uma camada específica, o Vue atual é apresentado oficialmente como um framework progressivo, acompanhado de um ecossistema que inclui soluções oficiais para roteamento, gerenciamento de estado, ferramentas de desenvolvimento e outras necessidades de aplicações maiores. O Vue Router é a solução oficial para roteamento no lado do cliente, enquanto o Pinia é a recomendação atual para gerenciamento de estado em aplicações que precisam de uma solução dedicada.

3. Principais Características

3.1 Componentização

O Vue organiza a interface em componentes independentes e reutilizáveis. Cada componente pode concentrar estrutura, lógica e estilo de uma parte específica da aplicação. Essa organização permite dividir uma interface grande em partes menores, facilitando a manutenção, o reuso e o trabalho em equipe.

3.2 Sistema de Reatividade

A reatividade é um dos principais recursos do Vue. Quando um estado reativo é alterado, o framework identifica as mudanças necessárias e atualiza a interface correspondente. A API moderna oferece recursos como `ref()`, `reactive()`, `computed()` e `watch()`, permitindo representar o estado da aplicação de forma declarativa.

Esse mecanismo reduz a necessidade de manipulação manual do DOM pelo desenvolvedor. Em vez de determinar individualmente quais elementos da página precisam ser alterados, o

programador descreve a relação entre estado e interface, deixando o Vue responsável pela atualização reativa.

3.3 Templates e Single-File Components

O Vue permite criar componentes em arquivos com extensão `.vue`, conhecidos como Single-File Components (SFC). Neles, a estrutura visual pode ser definida em `<template>`, a lógica em `<script>` e os estilos em `<style>`. Essa abordagem mantém as partes relacionadas de um componente organizadas no mesmo arquivo e facilita a reutilização.

O recurso `<script setup>`, utilizado com a Composition API, torna a escrita de componentes mais direta e reduz código repetitivo. A documentação oficial apresenta o suporte a Single-File Components e `<script setup>` como parte do conjunto moderno de recursos do Vue.

3.4 Diretivas e Vinculação de Dados

As diretivas são atributos especiais utilizados nos templates para adicionar comportamentos à interface. Entre os exemplos mais conhecidos estão `v-if` para renderização condicional, `v-for` para renderização de listas, `v-on` ou `@` para eventos e `v-model` para vinculação de valores de formulários.

Esse modelo permite relacionar elementos da interface ao estado da aplicação de maneira declarativa. Assim, uma alteração no estado pode refletir automaticamente na tela, enquanto eventos de interação do usuário podem modificar esse estado.

3.5 Composition API

A Composition API é uma abordagem moderna para organizar a lógica dos componentes Vue. Ela fornece funções como `ref()`, `reactive()`, `computed()` e `watch()`, permitindo agrupar lógica relacionada mesmo quando ela pertence a diferentes partes da interface. Esse modelo também favorece a criação de funções reutilizáveis chamadas composables.

3.6 Ecossistema e Ferramentas

O ecossistema oficial do Vue inclui ferramentas que ampliam sua utilização em projetos reais. O Vue Router fornece roteamento para aplicações, enquanto o Pinia oferece gerenciamento de estado compartilhado. Para criação de projetos Vue utilizando Vite, o ecossistema oficial também disponibiliza o create-vue. [↗](#) [↗](#) [↗](#) [↗](#)

Recurso	Função	Importância no projeto
Vue Router	Gerencia a navegação e as rotas da aplicação.	Permite criar aplicações com múltiplas telas e URLs sem recarregar toda a página.
Pinia	Gerencia estado compartilhado entre componentes.	Facilita o controle de dados globais, como usuário logado, carrinho e preferências.
Vite / create-vue	Criação e desenvolvimento do projeto Vue.	Oferece um ambiente moderno e rápido para desenvolvimento Front-end.
Vue DevTools	Inspeção e depuração de aplicações Vue.	Ajuda a visualizar componentes, estado e eventos durante o desenvolvimento.
Nuxt	Framework baseado em Vue para	Podem ser utilizados para SSR,

	aplicações mais completas.	geração estática e aplicações Web de maior complexidade.
--	----------------------------	--

Tabela 1 — Principais ferramentas e tecnologias relacionadas ao ecossistema Vue.

4. Vantagens do Vue.js

Facilidade de aprendizado — A sintaxe é próxima de HTML, CSS e JavaScript, e a documentação oficial é organizada para facilitar a entrada de novos desenvolvedores.

Reatividade — O sistema reativo permite que mudanças no estado sejam refletidas na interface de maneira declarativa.

Componentização — Componentes independentes podem ser reutilizados em diferentes partes da aplicação.

Adoção incremental — O framework pode ser incorporado gradualmente a projetos existentes, sem exigir necessariamente a reconstrução completa da aplicação.

Ecossistema oficial — Soluções como Vue Router e Pinia oferecem caminhos oficiais para necessidades comuns de aplicações maiores.

Suporte a TypeScript — O Vue é implementado em TypeScript e oferece suporte de primeira classe à linguagem, embora seu uso não seja obrigatório.

Vantagem	Descrição
Alta produtividade	A sintaxe declarativa e a reatividade reduzem código manual para atualizar a interface.
Componentização	Permite dividir aplicações em componentes menores, reutilizáveis e fáceis de manter.
Reatividade	Alterações no estado podem atualizar automaticamente as partes necessárias da interface.
Flexibilidade	Pode ser usado em pequenos componentes ou em aplicações completas.
Ecossistema	Possui soluções oficiais para necessidades comuns como roteamento e gerenciamento de estado.
TypeScript	Possui suporte de primeira classe ao TypeScript para projetos que precisam de tipagem estática.
Adoção incremental	Pode ser incorporado gradualmente em aplicações existentes.

Tabela 2 — Principais vantagens do Vue.js.

Desempenho — O sistema de renderização é reativo e possui otimizações de compilação, reduzindo a necessidade de otimizações manuais em muitos cenários.

5. Desvantagens e Pontos de Atenção

- Ecossistema em evolução** — como outras tecnologias Front-end modernas, o Vue possui versões e ferramentas que evoluem continuamente, exigindo atualização dos conhecimentos.
- Arquitetura de projetos maiores** — aplicações grandes precisam de organização consistente de componentes, estado, rotas, testes e estrutura de pastas.

- Escolhas adicionais — embora o ecossistema ofereça soluções oficiais, projetos podem precisar de ferramentas complementares para testes, requisições HTTP, validação e outras necessidades.
- Curva de aprendizado inicial — conceitos como reatividade, Composition API, props, eventos, composables e gerenciamento de estado exigem prática para serem utilizados de forma adequada.
- Dependência de boas práticas — a flexibilidade do Vue facilita diferentes formas de estruturar um projeto, mas equipes precisam estabelecer padrões para manter consistência.

6. Aplicações no Mercado

O Vue.js é utilizado para construir interfaces Web de diferentes níveis de complexidade, desde componentes incorporados em páginas existentes até aplicações completas. A própria documentação destaca que o framework pode escalar de forma incremental e que é adequado para Single-Page Applications. [\[cite turn0search0 turn0search7\]](#)

Entre os principais tipos de sistemas que podem utilizar Vue estão dashboards administrativos, sistemas internos, plataformas SaaS, e-commerces, portais institucionais, sistemas de gerenciamento e aplicações interativas. O ecossistema oficial fornece recursos para componentes, roteamento, estado, SSR e outras necessidades de projetos de maior porte. [\[cite turn0search14 turn0search9\]](#)

Além disso, o projeto Vue mantém um repositório oficial ativo e disponibiliza ferramentas como Vue Router, Pinia, DevTools e create-vue, indicando a existência de um ecossistema estruturado para desenvolvimento e manutenção de aplicações. A documentação oficial também afirma que o Vue é utilizado em produção por organizações como Wikimedia Foundation, NASA, Apple, Google, Microsoft, GitLab, Zoom, Tencent, Weibo, Bilibili e Kuaishou. Isso evidencia que a tecnologia não está limitada a pequenos projetos ou aplicações acadêmicas, podendo ser encontrada em produtos e sistemas utilizados em escala. O GitLab, por exemplo, publicou que escolheu Vue por sua combinação de estrutura e simplicidade e por permitir resolver determinadas mudanças de interface com menos código em comparação à abordagem anterior baseada em jQuery.

Empresa / Organização	Uso relacionado ao Vue.js	Observação
GitLab	Interface e partes da aplicação Web	O GitLab publicou uma explicação sobre a escolha do Vue, destacando simplicidade, menor quantidade de código e atualização automática das views.
NASA	Aplicações e interfaces Web	A documentação oficial do Vue cita a NASA entre organizações que utilizam Vue em produção.
Apple	Aplicações Web	A documentação oficial do Vue cita a Apple entre organizações que utilizam Vue em produção.
Google	Aplicações e plataformas Web	A documentação oficial do Vue cita o Google entre organizações que utilizam Vue em produção.
Microsoft	Aplicações Web	A documentação oficial do Vue cita a Microsoft entre organizações que utilizam Vue em produção.

Zoom	Aplicações Web	A documentação oficial do Vue cita o Zoom entre organizações que utilizam Vue em produção.
Tencent	Aplicações Web	A documentação oficial do Vue cita a Tencent entre organizações que utilizam Vue em produção.
Weibo	Plataformas Web	A documentação oficial do Vue cita o Weibo entre organizações que utilizam Vue em produção.
Bilibili	Plataformas Web	A documentação oficial do Vue cita o Bilibili entre organizações que utilizam Vue em produção.

Tabela 3 — Exemplos de organizações citadas pela documentação oficial do Vue como usuárias da tecnologia em produção.

7. Exemplo Prático de Utilização em um Projeto Web

Para demonstrar a aplicação prática dos conceitos apresentados, o grupo desenvolveu um exemplo simples de componente Vue: uma Lista de Tarefas (To-Do List). A funcionalidade ilustra o uso de componentes, estado reativo, eventos, diretivas e renderização dinâmica.

7.1 Estrutura do Projeto

Um projeto Vue moderno criado com Vite pode seguir uma organização semelhante à apresentada abaixo:

```

meu-projeto-vue/
├── node_modules/
├── public/
├── src/
│   ├── components/
│   │   └── ListaTarefas.vue
│   ├── App.vue
│   └── main.js
├── index.html
├── package.json
└── vite.config.js

```

Figura 2 — Estrutura de pastas típica de um projeto Vue criado com Vite.

7.2 Componente de Lista de Tarefas (To-Do List)

Arquivo: src/components/ListaTarefas.vue

```

<script setup>
import { ref } from 'vue'

const tarefas = ref([])
const novaTarefa = ref('')

```

```

function adicionarTarefa() {
  if (novaTarefa.value.trim() === '') return
  tarefas.value.push(novaTarefa.value)
  novaTarefa.value = ''
}

function removerTarefa(index) {
  tarefas.value.splice(index, 1)
}
</script>

<template>
  <div class="lista-tarefas">
    <h2>Minhas Tarefas</h2>

    <input
      v-model="novaTarefa"
      type="text"
      placeholder="Digite uma nova tarefa"
    />

    <button @click="adicionarTarefa">Adicionar</button>

    <ul>
      <li v-for="(tarefa, index) in tarefas" :key="index">
        {{ tarefa }}
        <button @click="removerTarefa(index)">Remover</button>
      </li>
    </ul>
  </div>
</template>

```

Figura 3 — Componente funcional ListaTarefas utilizando ref(), v-model, eventos e v-for.

Arquivo: src/App.vue

```

<script setup>
import ListaTarefas from './components/ListaTarefas.vue'
</script>

<template>
  <div class="App">

```



```
        <ListaTarefas />
    </div>
</template>
```

Figura 4 — Componente principal App.vue que renderiza o componente ListaTarefas.

7.3 Explicação do Código

- `ref()`: é utilizado para criar referências reativas que armazenam a lista de tarefas e o texto digitado pelo usuário.
- Componentização: a funcionalidade foi isolada no componente `ListaTarefas.vue` e depois importada pelo `App.vue`, facilitando organização e reuso.
- `v-model`: cria uma vinculação entre o campo de entrada e a variável `novaTarefa`, permitindo que o estado acompanhe o valor digitado.
- Eventos: o modificador `@click` conecta os botões às funções responsáveis por adicionar e remover tarefas.
- `v-for`: percorre a lista reativa e cria dinamicamente um elemento `<li>` para cada tarefa cadastrada.
- Renderização reativa: quando o array de tarefas é alterado, o Vue atualiza a interface relacionada automaticamente.

Esse exemplo, embora simples, representa a base de funcionalidades presentes em sistemas reais, como painéis de gerenciamento de tarefas, carrinhos de compra em e-commerces, cadastros e formulários dinâmicos. Os mesmos princípios de estado, componentes e eventos podem ser ampliados para aplicações Web mais complexas.

8. Conclusão

A partir do estudo realizado, o grupo pôde compreender que o Vue.js é uma alternativa relevante para o desenvolvimento de interfaces Web modernas. Seus principais recursos incluem componentização, sistema de reatividade, templates declarativos, Composition API e um ecossistema oficial que oferece soluções para roteamento e gerenciamento de estado.

O exemplo prático desenvolvido — uma lista de tarefas — permitiu visualizar como os conceitos teóricos podem ser aplicados na construção de uma funcionalidade real. O uso de `ref()`, `v-model`, `v-for`, eventos e componentes demonstrou como o Vue simplifica a ligação entre estado e interface.

Assim, conclui-se que o Vue.js representa uma tecnologia importante para a formação em Desenvolvimento Web, especialmente por combinar uma abordagem progressiva com recursos suficientes para aplicações de diferentes portes. O conhecimento de seu ecossistema também contribui para a preparação dos integrantes do grupo para projetos profissionais de Front-end.

9. Referências

VUE.JS. Documentação oficial do Vue.js. Disponível em: <https://vuejs.org/>. Acesso em: ago. 2026.

VUE.JS. Vue Router — Documentação oficial. Disponível em: <https://router.vuejs.org/>. Acesso em: ago. 2026.

VUE.JS. State Management — Documentação oficial. Disponível em: <https://vuejs.org/guide/scaling-up/state-management.html>. Acesso em: ago. 2026.

VUEJS. Vue.js — Repositório oficial no GitHub. Disponível em: <https://github.com/vuejs/core>. Acesso em: ago. 2026.

VUE.JS. API Reference — Documentação oficial. Disponível em: <https://vuejs.org/api/>. Acesso em: ago. 2026.

Observação sobre as referências: As informações sobre empresas foram baseadas principalmente na documentação oficial do Vue.js e em materiais publicados pelas próprias organizações. A lista de empresas deve ser interpretada como exemplos de uso da tecnologia, pois uma mesma empresa pode utilizar Vue apenas em determinadas aplicações, páginas ou partes de seu produto.